

# DBMS PLACEMENT PREPARATION NOTES

Modules 6–9 • SQL Basics • SQL Querying • SQL Functions • Joins

**Study goal:** Understand the terminology first, then memorize syntax and solve placement-style queries.

**Efftronics focus:** DBMS is explicitly listed in the Software Engineer assessment areas, alongside programming, algorithms/data structures, software architecture, OOD, computer architecture and OS. ■filecite■turn0file0■L59-L80■

## MODULE 6 — SQL BASICS

**SQL (Structured Query Language):** Language used to create, access, modify, and control data in relational databases.

### 1. SQL Command Categories

| Category | Meaning                             | Main Commands                 |
|----------|-------------------------------------|-------------------------------|
| DDL      | Defines/changes database structure. | CREATE, ALTER, DROP, TRUNCATE |
| DML      | Changes data in tables.             | INSERT, UPDATE, DELETE        |
| DQL      | Retrieves data.                     | SELECT                        |
| DCL      | Controls permissions.               | GRANT, REVOKE                 |
| TCL      | Controls transactions.              | COMMIT, ROLLBACK, SAVEPOINT   |

### 2. DDL — Data Definition Language

**CREATE:** Creates a database object. **ALTER:** Changes its structure. **DROP:** Removes the object. **TRUNCATE:** Removes all rows but keeps the table structure.

```
CREATE TABLE Student (id INT PRIMARY KEY, name VARCHAR(50), marks INT);  
ALTER TABLE Student ADD email VARCHAR(100);
```

### 3. DML — Data Manipulation Language

**INSERT:** Adds rows. **UPDATE:** Changes existing rows. **DELETE:** Removes rows.

```
INSERT INTO Student VALUES (1, 'Ravi', 85);  
UPDATE Student SET marks = 90 WHERE id = 1;  
DELETE FROM Student WHERE id = 1;
```

### 4. DQL — Data Query Language

**SELECT:** Retrieves data from one or more tables.

```
SELECT name, marks FROM Student WHERE marks > 80;
```

### 5. DCL — Data Control Language

**GRANT:** Gives privileges. **REVOKE:** Removes privileges.

### 6. TCL — Transaction Control Language

| Command   | Meaning                                    |
|-----------|--------------------------------------------|
| COMMIT    | Permanently saves the transaction changes. |
| ROLLBACK  | Undoes uncommitted changes.                |
| SAVEPOINT | Creates a point for partial rollback.      |

### 7. DELETE vs TRUNCATE vs DROP

| DELETE             | TRUNCATE              | DROP                        |
|--------------------|-----------------------|-----------------------------|
| Removes rows.      | Removes all rows.     | Removes the table/object.   |
| WHERE can be used. | WHERE cannot be used. | Table structure is removed. |
| Table remains.     | Table remains.        | Table no longer exists.     |

■ Memory: **DELETE** → rows | **TRUNCATE** → all rows | **DROP** → table/object

## MODULE 7 — SQL QUERYING

**Query:** A request to retrieve or manipulate data in a database.

Basic pattern:

```
SELECT column1, column2 FROM table WHERE condition;
```

### 1. SELECT

Retrieves selected columns. \* means all columns.

```
SELECT name, marks FROM Student;
```

### 2. DISTINCT

Removes duplicate values from the result.

```
SELECT DISTINCT department FROM Employee;
```

### 3. WHERE

Filters rows that satisfy a condition.

```
SELECT * FROM Employee WHERE salary > 50000;
```

### 4. Comparison & Logical Operators

| Operator     | Meaning                        |
|--------------|--------------------------------|
| =            | Equal                          |
| <> / !=      | Not equal                      |
| >, <, >=, <= | Comparison                     |
| AND          | All conditions must be true    |
| OR           | At least one condition is true |
| NOT          | Negates a condition            |

### 5. BETWEEN

Checks whether a value is within an inclusive range.

```
SELECT * FROM Employee WHERE salary BETWEEN 30000 AND 60000;
```

### 6. IN / NOT IN

Checks whether a value is or is not in a list.

```
SELECT * FROM Employee WHERE department IN ('IT', 'HR');
```

### 7. LIKE

Pattern matching for text.

| Pattern | Meaning       |
|---------|---------------|
| 'A%'    | Starts with A |
| '%A'    | Ends with A   |

| Pattern | Meaning               |
|---------|-----------------------|
| '%A%'   | Contains A            |
| '_A%'   | Second character is A |

## 8. NULL

**NULL**: Missing/unknown value. Use **IS NULL** or **IS NOT NULL**, not = NULL.

```
SELECT * FROM Employee WHERE manager_id IS NULL;
```

## 9. ORDER BY

Sorts results. **ASC** = ascending; **DESC** = descending.

```
SELECT name, salary FROM Employee ORDER BY salary DESC;
```

## 10. GROUP BY

Groups rows having the same value, usually for aggregate functions.

```
SELECT department, COUNT(*) FROM Employee GROUP BY department;
```

## 11. HAVING

Filters groups after GROUP BY.

```
SELECT department, COUNT(*) FROM Employee GROUP BY department HAVING COUNT(*) > 5;
```

## 12. WHERE vs HAVING

| WHERE                   | HAVING                |
|-------------------------|-----------------------|
| Filters rows.           | Filters groups.       |
| Used before grouping.   | Used after grouping.  |
| Example: salary > 50000 | Example: COUNT(*) > 5 |

## 13. Common SQL Clause Order



*Common filtering / grouping flow*

Written query order: **SELECT** → **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **ORDER BY**.

## MODULE 8 — SQL FUNCTIONS

**Function:** A built-in operation that takes input and returns a result.

### 1. Aggregate Functions

Operate on multiple rows and return one result per group.

| Function | Purpose             |
|----------|---------------------|
| COUNT()  | Counts rows/values  |
| SUM()    | Adds numeric values |
| AVG()    | Calculates average  |
| MIN()    | Finds minimum       |
| MAX()    | Finds maximum       |

```
SELECT COUNT(*) FROM Employee;
```

```
SELECT department, AVG(salary) FROM Employee GROUP BY department;
```

```
SELECT MAX(salary) FROM Employee;
```

### 2. COUNT(\*) vs COUNT(column)

**COUNT(\*)** counts rows. **COUNT(column)** counts only non-NULL values in that column.

### 3. String Functions

| Function    | Purpose                    |
|-------------|----------------------------|
| UPPER()     | Converts text to uppercase |
| LOWER()     | Converts text to lowercase |
| LENGTH()    | Returns string length      |
| CONCAT()    | Joins strings              |
| SUBSTRING() | Extracts part of a string  |

### 4. Numeric Functions

| Function | Purpose         |
|----------|-----------------|
| ROUND()  | Rounds a number |
| CEIL()   | Rounds upward   |
| FLOOR()  | Rounds downward |
| ABS()    | Absolute value  |

### 5. Date Functions

Used to work with dates and times. Exact function names can vary by DBMS.

Placement focus: current date/time, extracting year/month/day, and calculating date differences.

### 6. Aggregate vs Scalar Functions

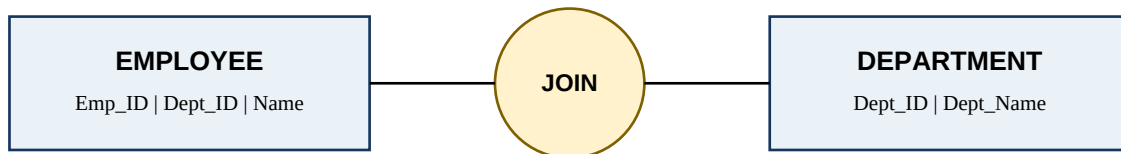
| Aggregate                     | Scalar                               |
|-------------------------------|--------------------------------------|
| Works on multiple rows.       | Works on one value at a time.        |
| Returns one result per group. | Returns a result for each input row. |
| COUNT, SUM, AVG, MIN, MAX     | UPPER, LOWER, LENGTH, ROUND          |

### ■ Placement traps

- COUNT(\*) counts rows even if individual columns contain NULL.
- COUNT(column) ignores NULL values in that column.
- Aggregate functions are commonly combined with GROUP BY.

## MODULE 9 — SQL JOINS

**Join:** Combines rows from two or more tables using a related column or condition.



*EMPLOYEE.Dept\_ID = DEPARTMENT.Dept\_ID*

### Example tables

| EMPLOYEE   | Name      | Dept_ID |
|------------|-----------|---------|
| 1          | Ravi      | 10      |
| 2          | Anu       | 20      |
| 3          | Kiran     | 30      |
| DEPARTMENT | Dept_Name |         |
| 10         | IT        |         |
| 20         | HR        |         |
| 40         | Sales     |         |

### 1. INNER JOIN

Returns only rows with a match in both tables.

```
SELECT e.name, d.dept_name FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;
```

### 2. LEFT JOIN

Returns every row from the left table plus matching rows from the right. Missing right-side values become NULL.

### 3. RIGHT JOIN

Returns every row from the right table plus matching rows from the left. Missing left-side values become NULL.

### 4. FULL OUTER JOIN

Returns all matching and non-matching rows from both tables. Unmatched values become NULL.

### 5. SELF JOIN

Joins a table with itself. Common example: employee → manager.

### 6. CROSS JOIN

Returns the Cartesian product: every row of the first table paired with every row of the second table.

### 7. EQUI JOIN

A join whose condition uses equality (=).

## 8. NON-EQUI JOIN

A join whose condition uses operators other than equality, such as >, <, or BETWEEN.

## 9. NATURAL JOIN

Automatically joins columns with the same name and compatible data types. It should be used carefully because the matching columns are selected automatically.

## 10. Join Comparison

| Join       | Main idea                                          |
|------------|----------------------------------------------------|
| INNER      | Only matching rows                                 |
| LEFT       | All left rows + matching right                     |
| RIGHT      | All right rows + matching left                     |
| FULL OUTER | All rows from both sides                           |
| SELF       | Table joined with itself                           |
| CROSS      | Every combination of rows                          |
| EQUI       | Condition uses =                                   |
| NON-EQUI   | Condition uses other operators                     |
| NATURAL    | Automatically matches same-name compatible columns |

## 11. Standard Join Pattern

```
SELECT columns FROM TableA A JOIN TableB B ON A.key = B.key;
```

## 12. Placement Problems to Practice

- Find employees with their department names.
- Find employees who do not belong to any department.
- Find departments that have no employees.
- Find employees and their managers using SELF JOIN.
- Find all employees and show NULL when their department is missing.
- Find matching records between two tables.
- Find every possible employee-department combination using CROSS JOIN.

## ■ 1-MINUTE REVISION

| Topic                 | Remember                                  |
|-----------------------|-------------------------------------------|
| DDL                   | Structure: CREATE, ALTER, DROP, TRUNCATE  |
| DML                   | Data changes: INSERT, UPDATE, DELETE      |
| DQL                   | Retrieval: SELECT                         |
| DCL                   | Permissions: GRANT, REVOKE                |
| TCL                   | Transactions: COMMIT, ROLLBACK, SAVEPOINT |
| WHERE                 | Filters rows                              |
| GROUP BY              | Creates groups                            |
| HAVING                | Filters groups                            |
| ORDER BY              | Sorts results                             |
| COUNT/SUM/AVG/MIN/MAX | Aggregate functions                       |

| Topic      | Remember           |
|------------|--------------------|
| INNER JOIN | Matching rows      |
| LEFT JOIN  | All left + matches |
| SELF JOIN  | Table with itself  |
| CROSS JOIN | Cartesian product  |

**Practice rule:** Read the notes once, then write queries yourself. SQL becomes strong through repeated query writing, not memorization alone.